

LISTA DE EXERCÍCIOS 3.11

1. Explique brevemente o que vem a ser o alocador *slab*.

Resposta: o alocador *slab* é uma estratégia alternativa para alocação de memória em espaço de kernel. Utiliza um bloco de armazenamento (*slab*) formado por uma ou mais páginas fisicamente contíguas. *Slabs* são organizados em caches. Um único cache é usado para cada estrutura de dados única de kernel. Cada cache é preenchido com objetos, os quais representam instâncias da estrutura de dados representada. Ao se criar o cache, este é preenchido com objetos em posições do *slab* marcadas como livres (*free*). Quando estruturas são armazenadas, as posições nas quais os objetos armazenados são marcadas como usadas (*used*). Se o *slab* está cheio de objetos, o próximo objeto é alocado a partir de um *slab* vazio. Se não há *slabs* vazios, um novo *slab* é alocado. Benefícios para uso dessa estratégia incluem a não fragmentação e a possibilidade de satisfazer rapidamente uma requisição de memória.

2. Faça considerações sobre a pré-paginação e a redução de *page faults* em SOs.

Resposta: Para tratar o grande número de *page faults* que ocorre durante a carga inicial de processos, é possível pré-paginar todas ou algumas das páginas que um processo irá precisar, antes que elas sejam referenciadas. A questão fundamental refere-se a utilização ou não das páginas pré-paginadas. Caso não sejam utilizadas, as operações realizadas representam I/O e acesso a memória desperdiçados. Caso sejam utilizadas, deve-se estabelecer o custo entre *page faults* evitadas e o custo de pré-paginação.

3. Qual é o impacto de seleção de tamanho de páginas em SOs?

Resposta: A seleção de tamanho de páginas tem influência direta em questões relacionadas a fragmentação interna (*frames* maiores estão mais sujeitos a fragmentação interna enquanto *frames* menores possuem melhor resolução quanto a representação de dados com menor fragmentação interna), ao tamanho da tabela de páginas (*frames* menores possuem melhor resolução, mas aumentam o tamanho da tabela de páginas para representar a maior quantidade de *frames* em memória), a sobrecarga de I/O (tamanho de *frames* menores implicam em um número maior de operações de I/O), a uma maior quantidade de *page faults* (tamanho de *frames* menores implicam em uma maior quantidade de páginas em memória e um maior número de *page faults*, por exemplo, na carga inicial de processos), em referência de localidade (maior ou menor quantidade de páginas para representar um dado contexto em um processo), e finalmente, em relação ao tamanho da TLB, o qual está diretamente relacionado a quantidade de páginas representada (para tamanhos de *frames* maiores, a TLB seria capaz de representar para cada página uma maior quantidade de dados em memória) e a maior ou menor efetividade da mesma.

4. Discorra sobre a relevância de I/O *interlock* com relação a possibilidade de operações de I/O perdidas em caso de *swap out* de processo.

Resposta: I/O Interlock refere-se ao bloqueio de páginas na memória com o intuito de se evitar sua remoção da memória por um algoritmo de substituição de página. Assim, faz-se a fixação destas na memória (*pinning*) pelo menos até que a operação de I/O seja encerrada. Posteriormente, o algoritmo de substituição de página em uso pode retomar sua operação levando em conta o processo que utiliza estas páginas. A operação de *pinning* pode ser definitiva ou não para uma determinada região de memória do processo, dependendo de questões relacionadas ao privilégio da aplicação quanto a execução da chamada de sistema apropriada para a realização da operação de *pinning*.